

Lab 4 Build Guide: CI/CD Pipeline for Network Changes

This guide walks through building a complete CI/CD pipeline that validates, plans, and deploys network changes automatically. By the end you will have GitHub Actions workflows that gate every change through the validation framework from Lab 2, generate config diffs for review, and push approved changes to the fabric via Scrapli.

What We Built

Labs 1 through 3 created a validated data model and a config generation engine. But the workflow is still manual: you edit YAML, run the validator, run the generator, and somehow get the configs onto the routers. Lab 4 automates that entire chain.

The pipeline has three stages mapped to the Git workflow. Pushing to a feature branch triggers validation. Opening a pull request triggers config generation with a diff posted as a comment. Merging to main triggers deployment. At no point does a human SSH into a router and type commands. The pipeline is the only path from intent to configuration.

The deployment script uses Scrapli, an async SSH library purpose-built for network devices. It connects to all six FRR containers concurrently and pushes the generated configs. Credentials come from environment variables, never from code.

Prerequisites

You need Labs 1-3 complete with all validation passing and configs generating correctly. You also need the ContainerLab topology running on the VM.

```
uv run python validate.py
uv run python -m generators.python.render
sudo containerlab inspect --topo containerlab/topology.yaml
```

All 39 checks pass, 6 configs generate, and all 6 nodes are running.

Part 1: ContainerLab Topology Update

Before we can deploy configs, the ContainerLab topology needs a small update. FRR's default container image has ospfd and bgpd disabled. Rather than restarting FRR after deployment (which destroys the veth interfaces ContainerLab creates), we mount a custom daemons file that enables both daemons at startup.

The file is at containerlab/frr_daemons:

```
cat containerlab/frr_daemons
```

It is the same as FRR's default daemons file but with bgpd=yes and ospfd=yes. The topology.yaml references it through a binds directive that mounts it into every container at /etc/frr/daemons.

If your ContainerLab topology is already running, destroy and redeploy to pick up the new daemons file:

```
sudo containerlab destroy --topo containerlab/topology.yaml
sudo containerlab deploy --topo containerlab/topology.yaml
```

Verify that ospfd and bgpd are running in the containers:

```
docker exec clab-nac-spine-leaf-spine1 ps aux | grep -E 'ospf|bgp'
```

You should see both /usr/lib/frr/ospfd and /usr/lib/frr/bgpd in the process list.

Part 2: The Deployment Script

The deployment script lives at `deploy/scrapli_deploy.py`. It reads the generated configs from `configs/` and pushes them to the FRR containers using `docker exec`.

```
cat deploy/scrapli_deploy.py
```

How It Works

The deployment has two phases. Phase 1 handles the FRR control plane: the script loads the data model to get the list of devices, reads the corresponding config file from the `configs` directory, writes it into the container, and loads it through `vytysh -f`, which applies the configuration to the running daemons without restarting anything. Phase 2 handles the VXLAN dataplane: for each leaf and border device, the script generates and runs Linux networking commands that create VRF devices, VXLAN tunnel interfaces, bridges, and anycast gateway SVIs. Spines skip Phase 2 because they do not host VNIs.

The dataplane commands are derived from the same data model that drives config generation. The script determines which VRFs and network segments belong to each device using the same logic as the render engine (borders get everything, leafs get VRFs matching their assigned VLANs). For each VRF it creates a Linux VRF device with a routing table, an L3VNI VXLAN interface for inter-subnet routing, and a bridge to link them. For each network segment it creates an L2VNI VXLAN interface, a per-VLAN bridge, and adds the anycast gateway IP to the bridge.

The deployments are concurrent. All six devices are contacted at the same time using `asyncio`. The script uses `docker exec` because ContainerLab containers run on the same host and do not have SSH daemons. For production deployments to real network devices with SSH access, you would replace the `docker exec` calls with Scrapli SSH connections. The script structure (`async`, `concurrent`, `dry-run` support) stays the same.

Why `docker exec` Instead of SSH

FRR containers in ContainerLab are minimal Linux containers. They run FRR daemons but no SSH server. This is actually the common pattern for containerized network functions. The deployment script adapts to the environment: `docker exec` for lab containers, Scrapli SSH for production devices. The important thing is that the config generation and validation are identical regardless of how the config gets onto the device.

Dry Run

Always start with a dry run. This validates that all config files exist, are readable, and that the containers are reachable:

```
uv run python -m deploy.scrapli_deploy --dry-run
```

You should see a table with all six devices showing "OK" status, the number of config lines, and confirmation that each container is reachable.

Live Deployment

Make sure the ContainerLab topology is running, then deploy:

```
uv run python -m deploy.scrapli_deploy
```

The script writes each config into its container, loads it through `vytysh`, saves it to the startup configuration, and provisions the VXLAN dataplane. Leaf and border devices will show "dataplane provisioned" in their status message. Spines only show "daemons restarted" because they do not host VNIs. If any device fails, the script reports which one and why.

Verify the Deployment

Give OSPF about 20 seconds to converge, then check the routing state. OSPF needs to form adjacencies before BGP can establish sessions over the loopback addresses.

Check OSPF neighbors on spine1:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "show ip ospf neighbor"
```

You should see four neighbors in Full state: leaf1, leaf2, border1, and border2. Each one is connected over a point-to-point /31 link.

Check BGP summary on spine1:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "show bgp summary"
```

You should see five peers (spine2, leaf1, leaf2, border1, border2) in Established state with messages being exchanged. The IPv4 Unicast summary shows PfxRcvd values: spine2 sends 5 (all loopbacks), while each leaf/border sends 1 (their own loopback). The L2VPN EVPN summary shows EVPN prefixes: leafs send 2-3 Type-3 routes (one per VNI), borders send around 10 (Type-3 for all VNIs plus Type-5 IP prefix routes).

If BGP peers show Connect instead of Established, OSPF has not finished propagating loopback routes yet. Wait another 30 seconds and check again. BGP peers use loopback addresses as their update source, so the underlay must be converged first.

Check the BGP IPv4 unicast table to see all six loopbacks being advertised:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "show bgp ipv4 unicast"
```

You should see 6 routes and 10 total paths. Each device's loopback /32 is present, with multipath entries showing routes via both spine1 (local) and spine2 (reflected).

Check the EVPN table to see the overlay routes:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "show bgp l2vpn evpn"
```

You should see around 26 EVPN prefixes. Type-3 (IMET) routes show each VTEP advertising which VNIs it participates in. Type-5 (IP Prefix) routes show VRF subnets being advertised by the border devices with their router MACs for inter-subnet routing.

Verify the VXLAN dataplane on a leaf:

```
docker exec clab-nac-spine-leaf-leaf1 ip link show type vrf
docker exec clab-nac-spine-leaf-leaf1 ip link show type vxlan
docker exec clab-nac-spine-leaf-leaf1 ip route show vrf PRODUCTION
```

You should see the PRODUCTION VRF device in UP state, three VXLAN interfaces (the L3VNI for inter-subnet routing and one L2VNI per network segment), and connected routes for the SVI subnets in the VRF routing table.

This is the moment that ties the entire series together. The data model from Lab 1, validated by Lab 2, generated into configs by Lab 3, is now running as live routing protocol state on six FRR routers. OSPF adjacencies are up, BGP sessions are established, EVPN routes are being exchanged through the route reflectors, and the VXLAN dataplane is provisioned with VRF isolation and anycast gateways. Every bit of it came from YAML files in a Git repository.

Part 2: GitHub Actions Workflows

The pipeline is implemented as three GitHub Actions workflow files in .github/workflows/. Each one triggers on a different Git event.

Validate Workflow (Feature Branch Push)

```
cat .github/workflows/validate.yaml
```

This workflow triggers on any push to a feature/ or hotfix/ branch that modifies data model files, schemas, or validators. It installs uv, syncs dependencies, runs the four-layer validation, and runs the pytest suite. If any check fails, the workflow fails and the branch shows a red X in GitHub.

The workflow also generates an HTML validation report and uploads it as a build artifact. Reviewers can download it from the Actions tab to see the full test results.

Plan Workflow (Pull Request)

```
cat .github/workflows/plan.yaml
```

This workflow triggers when a pull request targets main. It runs all validation (same as the validate workflow), then generates configs from both the PR branch and the main branch. It diffs the two sets of configs and posts the result as a PR comment.

The PR comment shows a per-device diff in unified format. If you changed the BGP keepalive timer in the data model, the comment shows timers bgp 30 90 changing to timers bgp 60 90 on every device. Reviewers see exactly what will happen on the network before they approve.

On the first PR (when main has no generated configs yet), the comment lists all configs as new with their line counts.

Deploy Workflow (Merge to Main)

```
cat .github/workflows/deploy.yaml
```

This workflow triggers when changes are pushed to main (which happens on PR merge). It runs validation, generates configs, does a dry run, and then deploys via Scrapli.

This workflow uses runs-on: self-hosted instead of runs-on: ubuntu-latest. GitHub's hosted runners cannot reach your ContainerLab devices because they are on your local network. You need a self-hosted runner on the VM itself (or on a machine that has network access to the ContainerLab management subnet).

Setting Up a Self-Hosted Runner (Optional for Demo)

Setting up the self-hosted runner is optional for the initial demo. You can run the deployment manually with `uv run python -m deploy.scrapli_deploy` and still demonstrate the full workflow. When you are ready to fully automate it:

1. Go to your repo on GitHub: Settings > Actions > Runners > New self-hosted runner

2. Follow

3. Register it with the default labels or add clab-runner as a custom label

4. Start th

The runner needs to be on a machine that can reach the ContainerLab management network (172.20.20.0/24).

Part 3: The Branching Strategy

The pipeline enforces a branching strategy that maps directly to the three workflow triggers:

main is the production branch. It represents the current intended state of the network. Only pull requests can merge into main, and they must pass validation first. Every merge to main triggers a deployment.

feature/* branches are where changes happen. Each change gets its own branch: feature/add-vrf-staging, feature/update-bgp-timers, feature/new-leaf-switch. Pushing to a feature branch triggers validation immediately.

hotfix/* branches are for emergency changes. They follow the same validation pipeline but signal to reviewers that this is urgent and should be reviewed quickly.

Demo: The Full Workflow

This is the demo that ties the entire lab together. Walk through a complete change from branch to deployment.

Step 1: Create a Feature Branch

```
git checkout -b feature/update-bgp-timers
```

Step 2: Make a Data Model Change

Edit data/overlay.yaml and change the BGP keepalive from 30 to 20:

```
bgp:
  address_families:
    - evpn
    - ipv4_unicast
  timers:
    keepalive: 20    # was 30
    holdtime: 90
```

Also update data/defaults.yaml to match:

```
timers:
  ospf_hello: 10
  ospf_dead: 40
  bgp_keepalive: 20    # was 30
  bgp_holdtime: 90
```

Step 3: Validate Locally

```
uv run python validate.py
```

All 39 checks pass. The compliance rule CMP-06 still passes because holdtime (90) is $\geq 3 \times$ keepalive ($20 \times 3 = 60$).

Step 4: Regenerate Configs

```
uv run python -m generators.python.render
```

Check the diff:

```
grep "timers bgp" configs/spine1.conf
```

Should show timers bgp 20 90 instead of the old timers bgp 30 90.

Step 5: Commit and Push

```
git add data/overlay.yaml data/defaults.yaml
git commit -m "Update BGP keepalive from 30s to 20s"
git push origin feature/update-bgp-timers
```

If the validate workflow is set up, it runs automatically on this push.

Step 6: Open a Pull Request

```
gh pr create --title "Update BGP keepalive timer to 20s" --body "Reducing keepalive from 30s to 20s for faster convergen
```

Or create the PR through the GitHub web UI. If the plan workflow is set up, it posts a comment showing the config diff on all six devices.

Step 7: Review and Merge

The PR shows the data model diff (two lines changed in overlay.yaml and defaults.yaml) and the plan comment

shows the config impact (every device's timers bgp line changes). A reviewer approves, you merge, and if the deploy workflow and self-hosted runner are set up, the new configs are pushed automatically.

Step 8: Deploy Manually (If No Runner)

If you have not set up the self-hosted runner yet, deploy from the main branch manually:

```
git checkout main
git pull
uv run python -m generators.python.render
uv run python -m deploy.scrapli_deploy
```

Step 9: Verify

```
docker exec -it clab-nac-spine-leaf-spine1 vtysh -c "show running-config" | grep "timers bgp"
```

Should show timers bgp 20 90.

Step 10: Revert (If Desired)

If you want to put the timers back to 30/90 for the next lab:

```
git checkout -b feature/revert-bgp-timers
```

Edit the files back, validate, commit, merge. Same workflow.

Part 4: Commit the Pipeline

```
git checkout main
git status
```

You should see the new `deploy/` directory, `.github/workflows/`, updated `containerlab/` (`topology.yaml` and `frr_daemons`), updated `generators/python/templates/` (interface and VRF template fixes), and updated `pyproject.toml` and `uv.lock` (with Scrapli and asyncssh added).

```
git add deploy/ .github/workflows/ containerlab/ generators/python/templates/ pyproject.toml uv.lock docs/lab4-build-gui
git commit -m "Lab 4: CI/CD pipeline with deployment script and GitHub Actions"
```

Check the log:

```
git log --oneline
```

Four commits. Data model, validation, config generation, and now the CI/CD pipeline. The automation stack is complete from intent to deployment.

Part 5: What We Proved

By the end of this lab you have demonstrated three things.

First, that the entire path from data model change to running configuration can be automated. An engineer edits YAML, pushes a branch, opens a PR, gets a config diff for review, and merges. The pipeline handles validation, generation, and deployment. At no point does anyone SSH into a router.

Second, that the pipeline is a safety net. Even if an engineer introduces a bad change, the validation stage catches it before it reaches the network. The plan stage shows exactly what will change. The review step gives a human the final say. This is the layered defense model that the Cisco NaC paper advocates.

Third, that the deployment itself is code. The deploy script is version-controlled, testable, and auditable. You can see exactly what the deployment does by reading the script. You can dry-run it before going live. You can trace any deployed config back to the commit that generated it.

Troubleshooting

Deployment says "container not running": Make sure the ContainerLab topology is running. Run `sudo containerlab inspect --topo containerlab/topology.yaml` to check. All six nodes need to be in "running" state.

OSPF neighbors not forming: Check that the ContainerLab topology was deployed with the custom `frr_daemons` file that enables `ospfd`. Run `docker exec clab-nac-spine-leaf-spine1 ps aux | grep ospfd` to verify. If `ospfd` is not running, destroy and redeploy ContainerLab to pick up the `daemons` bind mount.

BGP peers stuck in Connect state: This means OSPF has not converged yet and the loopback routes are not reachable. Wait 30 seconds after deployment for OSPF to propagate loopback /32 routes. Check `docker exec clab-nac-spine-leaf-spine1 vtysh -c 'show ip route'` to see if peer loopbacks appear as OSPF routes.

FRR interfaces show as "down" after deployment: This happens if FRR was restarted during deployment, which destroys ContainerLab's veth interfaces. The deploy script avoids this by loading config via `vtysh -f` without restarting. If you see this, destroy and redeploy ContainerLab to recreate the interfaces, then run the deploy script again.

vtysh warnings about "unknown command": Some config lines (like comments starting with `!`) produce benign warnings. The deploy script filters these and only reports actual errors. If you see "unknown command" for real config lines, check that you are running FRR 10.3.1.

GitHub Actions workflow not triggering: Check that the branch name matches the pattern (feature/ or hotfix/). Also check that the changed files match the paths filter in the workflow. Changes to files outside the listed paths do not trigger the workflow.

Deploy workflow needs self-hosted runner: The deploy workflow uses `runs-on: self-hosted`. If you do not have a self-hosted runner registered, the job will sit in "queued" state forever. Run the deployment manually until the runner is set up.

PR comment not appearing: The plan workflow needs the `pull-requests: write` permission. Check that the workflow has the permissions block. Also check the Actions tab for errors in the "Post plan as PR comment" step.